Breakout Lab 3.2

Developer Orchestration Track

30 minutes | Independent or Groups of two

KavinSchool

Mission

Instead of Python orchestration, use the `claude-helper` toolkit to orchestrate agents in a standard developer workflow. You will build and test a "Hair Salon Login Flow" on top of the Password Strength Checker project.

STRATEGY

Model Selection

Use Haiku for codebase mapping, and Sonnet for code implementation.

ALLOWED STACK

Tools

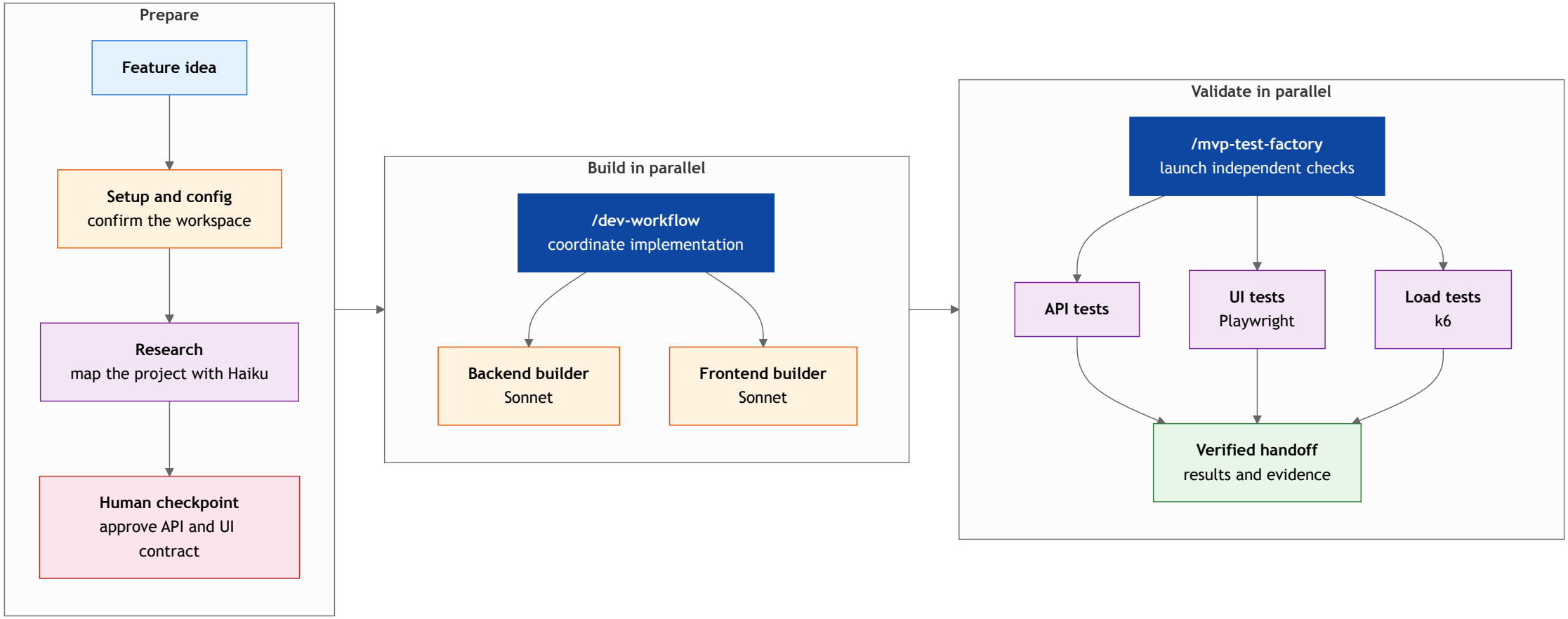
`/dev-workflow`, `/mvp-test-factory`, and Git worktrees.

REQUIRED EVIDENCE

Output

A working login feature and passing parallel tests (API, UI, Load).

The Developer Factory Chain



See `claude-helper/docs/developer-factory-chain.md` for the full breakdown.

Interactive: Dev Factory Workflow

Explore the same chain as a traced, clickable diagram: idea → context collection → planning → test writing → implementation → test-factory hand-off, with the two human checkpoints (plan approval, quality review) called out.

Open `dev-factory-workflow.html` from `claude-helper/docs/interactive/` to play the trace and inspect each stage.

Create a Fresh Repository

From your terminal, prepare your workspace:

```
git clone https://github.com/kpassoubady/password-strength-checker.git hair-salon-workspace
cd hair-salon-workspace
npm run install:server
```

Open your IDE and ensure `claude-helper` is available.

30-Minute Run of Show

Time	Checkpoint
0-5	Clone repo and use Haiku to map codebase.
5-10	Define the Hair Salon API and UI contract.
10-20	Use `/dev-workflow` to build the feature (Sonnet).
20-25	Launch `/mvp-test-factory` for parallel validation.
25-30	Verify passing API, UI, and Load tests.

Step 1 & 2: Research and Contract

Use the Haiku model (`claude-credits` rules):

- Prompt the agent to map the current file structure.
- Define what folders and files need to change for a Hair Salon Login Flow.
- Manually review and commit the contract/specification to `docs/hair-salon-spec.md` .

Why Haiku? It's the cheapest, fastest model—ideal for reading and searching codebases.

Step 3: Implement Feature

Switch to the Sonnet model:

- Trigger the `/dev-workflow` skill.
- Build the backend API logic independently.
- Build the UI React/HTML frontend independently.
- Start your server and manually verify the new Hair Salon login flow.

This matches the "Parallel Builders" phase of our factory chain.

Step 4: Parallel Validation

Invoke the test factory:

```
/mvp-test-factory /absolute/path/to/hair-salon-workspace
```

- Allow the orchestrator to spin up specialized agents.
- **API Tests**: Asserting HTTP routes.
- **UI Tests**: Using Playwright for E2E.
- **Performance**: Validating latency with k6.

Definition of Done

If blocked, use your task card, review the developer factory diagram, or ask a peer.

IMPLEMENTATION

The Hair Salon login flow functions correctly locally.

COST EFFICIENCY

Haiku was correctly utilized for the research phase.

PARALLELISM

API and UI builds did not block each other.

TEST VALIDATION

The MVP test factory produced passing, layered test suites.

Return for the Course Wrap-Up

Bring your generated test reports and your model selection strategy.

KavinSchool